

PROGRAMMING IN HASKELL



Parte 2 - Primeiros Passos

Haskell

- Programas em Haskell são estruturados em módulos (modules)
- Módulos contêm declarações (declarations)
- Os tipos de declarações mais essenciais são as vinculações (bindings)
- Vinculações são declarações (declarations) que dão nomes a expressões (expressions)
- Expressões são termos construídos a partir de constantes literais e chamadas de funções
- Expressões são avaliadas, resultando em um valor

Glasgow Haskell Compiler

- GHC é a implementação mais tradicional de Haskell, e engloba um compilador e um interpretador;
- A natureza interativa do interpretador o torna adequado para o ensino e prototipação;
- GHC é distribuído gratuitamente:

www.haskell.org

Iniciando o GHCi

O interpretador pode ser iniciado a partir do terminal através do comando ghci:

```
$ ghci
```

```
GHCi, version X: http://www.haskell.org/ghc/  :? for help
```

```
Prelude>
```

O prompt GHCi > significa que o interpretador está pronto para avaliar expressões.

Por exemplo, ele pode ser utilizado como uma calculadora para avaliar expressões numéricas simples:

```
> 2+3*4
```

```
14
```

```
> (2+3)*4
```

```
20
```

```
> sqrt (3^2 + 4^2)
```

```
5.0
```

O Standard Prelude

Haskell é distribuído com uma grande biblioteca padrão de funções. Além de funções numéricas tradicionais tal como $+$ e $*$, a biblioteca também provê muitas funções úteis sobre listas.

- Selecionar o primeiro elemento da lista:

```
> head [1,2,3,4,5]  
1
```

- Remover o primeiro elemento da lista:

```
> tail [1,2,3,4,5]  
[2,3,4,5]
```

- Selecionar o n-ésimo elemento da lista:

```
> [1,2,3,4,5] !! 2  
3
```

- Selecionar os primeiros n elementos da lista:

```
> take 3 [1,2,3,4,5]  
[1,2,3]
```

- Remover os primeiros n elementos da lista:

```
> drop 3 [1,2,3,4,5]  
[4,5]
```

- Calcular o tamanho da lista:

```
> length [1,2,3,4,5]  
5
```

- Calcular o somatório de uma lista de números:

```
> sum [1,2,3,4,5]  
15
```


- Calcular o produto de uma lista de números:

```
> product [1,2,3,4,5]  
120
```

- Concatenar duas listas:

```
> [1,2,3] ++ [4,5]  
[1,2,3,4,5]
```

- Inverter uma lista:

```
> reverse [1,2,3,4,5]  
[5,4,3,2,1]
```

Aplicação de Funções

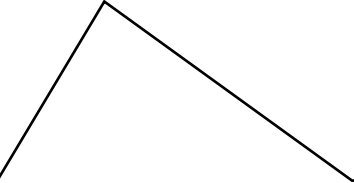
Na matemática, aplicação de funções é denotada via parênteses, e multiplicação é geralmente denotada utilizando a justaposição ou um espaçamento.

$$f(a,b) + c \ d$$

Aplica a função f sobre a e b , e adiciona o resultado ao produto de c e d .

Em Haskell, aplicação de função é denotada utilizando espaçamento, e multiplicação é denotada utilizando `*`.

```
f a b + c*d
```



Exemplo anterior, na sintaxe de Haskell.

Além disso, assume-se que aplicação de função possui maior prioridade que qualquer outro operador.

$f \ a \ + \ b$

Significa $f(a) + b$, ao invés de $f(a + b)$.

Exemplos

Matemática

$f(x)$

$f(x, y)$

$f(g(x))$

$f(x, g(y))$

$f(x)g(y)$

Haskell

$f\ x$

$f\ x\ y$

$f\ (g\ x)$

$f\ x\ (g\ y)$

$f\ x\ * \ g\ y$

Haskell Scripts

- Além das funções padrão, Haskell permite que nossas próprias funções sejam definidas;
- Novas funções são definidas dentro de um script, um arquivo texto contendo as definições;
- Por convenção, Haskell scripts usualmente possuem o sufixo .hs no nome do arquivo. Isso não é obrigatório, mas é uma forma útil de indicar o conteúdo do arquivo.

Meu Primeiro Script

Quando se desenvolve um script Haskell, é útil manter duas janelas abertas, uma para o editor do script, e outra para o GHCi.

Inicie um editor, digite as seguintes definições de funções, e armazene o arquivo como test.hs:

```
double x = x + x
```

```
quadruple x = double (double x)
```

Ainda com o editor aberto, em outra janela, inicie o GHCi com o novo script:

```
$ ghci test.hs
```

Agora, tanto a biblioteca padrão quanto o arquivo test.hs estão carregados, e funções de ambos podem ser utilizadas:

```
> quadruple 10  
40  
  
> take (double 2) [1,2,3,4,5,6]  
[1,2,3,4]
```


Sem fechar o GHCi, retorne ao editor, adicione as seguintes definições, e salve o arquivo:

```
factorial n = product [1..n]
average ns = sum ns `div` length ns
```

Observação:

- div está adornada com acentos graves;
- `x `f` y` é apenas açúcar sintático para `f x y`, indicando a aplicação infixa da função.

GHCi não detecta automaticamente as alterações em um script, o comando reload deve ser executado:

```
> :reload
Reading file "test.hs"

> factorial 10
3628800

> average [1,2,3,4,5]
3
```

Comandos GHCi

Comando

Significado

:load <i>name</i>	carrega script <i>name</i>
:reload	recarrega script atual
:set editor <i>name</i>	configura editor para <i>name</i>
:edit <i>name</i>	edita script <i>name</i>
:edit	edita script atual
:type <i>expr</i>	mostra o tipo de <i>expr</i>
:?	mostra todos os comandos
:quit	finaliza o GHCi

Convenções de Nomenclatura

- Nomes de funções e de argumentos devem iniciar com letra minúscula. Por exemplo:

myFun

fun1

arg_2

x'

- Argumentos do tipo lista usualmente tem um sufixo s no nome. Por exemplo:

xs

ns

nss

Regra de Layout

Em uma sequência de definições, cada definição deve iniciar precisamente na mesma coluna:

```
a = 10  
b = 20  
c = 30
```



```
a = 10  
    b = 20  
c = 30
```



```
a = 10  
b = 20  
    c = 30
```



A regra de layout evita a necessidade de sintaxe explícita para indicar agrupamento de definições.

```
a = b + c
  where
    b = 1
    c = 2
d = a * 2
```

agrupamento implícito



```
a = b + c
  where
    {b = 1;
     c = 2}
d = a * 2
```

agrupamento explícito

Exercícios

- (1) Experimente os slides 2-7 e 13-16 utilizando o GHCi.
- (2) Corrija os erros de sintaxe no programa a seguir, e teste sua solução no GHCi.

```
N = a 'div' length xs
  where
    a = 10
    xs = [1,2,3,4,5]
```

- (3) Mostre como a função last que seleciona o ultimo elemento de uma lista pode ser definida através de outras funções vistas em aula.
- (4) Você consegue pensar em outra definição?
- (5) De forma similar, mostre como a função init que remove o último elemento de uma lista pode ser definida de duas formas diferentes.